

SAXESS™

Exchange Message API (XM)

v 40.7.60.2, 2007-03-15 (2007-04-11 10:44)

This manual describes the Exchange Message API (XM API).

Copyright Notice

Copyright © 2006 OMX Technology AB

All rights reserved.

SAXESS™ is a trademark of OMX AB.

Disclaimer and Limitation of Liability

Whilst all reasonable care has been taken to ensure that the details are true and not misleading at the time of publication, no liability whatsoever is assumed by OMX AB, or any subsidiary of OMX AB, with respect to the accuracy or any use of the information provided herein.

Any license, delivery and support of software systems etc. require entering into separate agreements with OMX Technology AB.

This document contains confidential information and may not be modified or reproduced, in whole or in part, or transmitted in any form to any third party, without the written approval from OMX Technology AB.

Contents

1. Introduction
  1.1. Purpose
    1.1.1. XMAPI Environment
    1.1.2. Problems Solved with XMAPI
    1.1.3. Overview of the Message Flow
    1.1.4. Recovery Issues
  1.2. Failover
  1.3. XMAPI and Multithreaded Applications
  1.4. Terminology
  1.5. References
  1.6. Abbreviations
2. Type Definitions
  2.1. Constants
  2.2. Int32
  2.3. XM_Timespan
  2.4. XM_SessionMode
  2.5. XM_ErrorCode
  2.6. XMP_Type
  2.7. XMP_Tag
  2.8. XMP_CA
  2.9. XMP_EA
  2.10. XMP_SS
  2.11. XMP_PV
  2.12. XM_Service
  2.13. XM_Session
  2.14. XM_DC
  2.15. XM_Crypt
3. Function Interface
  3.1. Initialization
    3.1.1. XM_Init
    3.1.2. XM_InitEntropy
  3.2. Error Diagnostics
    3.2.1. XM_GetErrorMessage
  3.3. Service
    3.3.1. XM_Service_Init
    3.3.2. XM_Service_Destroy
    3.3.3. XM_Service_AddEntry
  3.4. Session
    3.4.1. XM_Session_ConnectService
    3.4.2. XM_Session_ConnectAddress
    3.4.3. XM_Session_Disconnect

3.4.4. XM_Session_GetAddress
3.4.5. XM_Session_TestDistribution
3.4.6. XM_Session_SetRecvTimeout
3.4.7. XM_Session_Send
3.4.8. XM_Session_Recv
3.4.9. XM_Session_MultiRecv
3.4.10. XM_Session_KeyExchange
3.4.11. XM_Session_SetRecvBufferSize
3.5. Decompression
3.5.1. XM_DC_Init
3.5.2. XM_DC_Destroy
3.5.3. XM_DC_Decompress
3.6. Encryption
3.6.1. XM_Crypt_Init
3.6.2. XM_Crypt_Destroy
3.6.3. XM_Crypt_Encrypt
3.6.4. XM_Crypt_Decrypt
3.7. XMP
3.7.1. XMP_GetBodyType
3.7.2. XMP_GetFrameLen
3.7.3. XMP_GetBody
3.7.4. XMP_GetNextFrame
3.7.5. XMP_GetBodyLen
3.7.6. XMP_GetExtHeaderLen
3.7.7. XMP_GetExtHdrEntry
3.7.8. XMP_AddExtHdrEntry
3.7.9. XMP_CreateMessage
3.7.10. XMP_SetHeader
Footnotes

1. Introduction

This document describes the current specification of the Exchange Message API (XMAPI).

XMAPI contains four sets of functions. By selecting the "Cooked" form of the session connect functions, the function sets for encryption/decryption and decompression will be called automatically for the application.

Each set is described later in this document:

TABLE 1

Function Set
Session handling
Encryption/Decryption
Decompression
XMP Handling

1.1. Purpose

1.1.1. XMAPI Environment

The API is delivered in source code form that should work on common platforms such as Posix and WIN32.

It compiles in at least the following environments: SPARC/Solaris/cc, SPARC/Solaris/CC, Intel/Linux/gcc, Intel/Linux/g++, Alpha/Tru64/cc, RS6000/AIX/xlc_r, HPPA/HPUX/cc and Intel/NT 4.x/Visual C++.

It should be easy to port to other platforms.

1.1.2. Problems Solved with XMAPI

The idea with XMAPI is to make it easier to access the SAXESS system. The user of XMAPI does not need to have knowledge about TCP/IP programming to connect to the central system. The user of XMAPI must however be familiar with the SAXESS Exchange Transaction Protocol (XTP) and its interaction model semantics. By using the XMP function set in XMAPI, the application can more easily implement the Exchange Message Protocol.

1.1.3. Overview of the Message Flow

The user of XMAPI must be familiar with the three different message flows in the SAXESS system, these are:

- Interactive Message Interface. The interactive message interface is a two-way transaction flow where the participant initiates all transactions. All interactive transactions are available by using the `XM_Session_Send` primitive to send a request and subsequently by receiving one or more replies using `XM_Session_Recv` or `XM_Session_MultiRecv`.
- Unicast Message Interface. The Unicast Message Interface is a two-way transaction flow. The participant sends logon, logoff and message request transactions by using `XM_Session_Send`. Information sent from SAXESS on this flow is private to the participant. Each participant has its own private message flow. Unicast messages are available by using the `XM_Session_Recv` primitive.
- Session-based Multicast Message Interface. The Multicast Message Interface is a two-way transaction flow that is used in the same way as the unicast interface.

1.1.4. Recovery Issues

All three message flows use TCP which is a reliable transport that guarantees that no gaps will occur in the sequence numbering of transactions. Catch-up of messages is performed automatically using the dissemination restart fid in the XTP logon message[1] (see the XTP protocol specification).

1.2. Failover

The `XM_Session_ConnectService` and `XM_Session_ConnectAddress` calls try all and one entry point respectively. In case the connection attempt fails, or if an existing session fails, the application should reconnect the service and logon again.

1.3. XMAPI and Multithreaded Applications

XMAPI is safe to use in multithreaded applications. Multiple threads may not simultaneously call XMAPI functions with the same XMAPI objects.

XM_Init should be called once before any other calls to XMAPI. If used, XM_InitEntropy must be called after XM_Init, but before calling any other XMAPI functions.

1.4. Terminology

The following describes some concepts used in this document:

- Frames and Messages. Frame and message is used interchangeably in this document. A frame is a complete XMP message, i.e. an XMP base header, extension header (0 or more entries) and body (if any).
- Body. The part of an XMP frame that follows the XMP base header and the extension header (if any).
- Raw and Cooked mode. XMAPI Sessions can be used in either of two modes. In Cooked mode XMAPI will handle decryption, encryption, decompression, and key exchange. In Raw mode the XMAPI users must handle these things themselves, either by calling XMAPI functions, and/or implementing the same functionality.

1.5. References

- Exchange Message Protocol (XMP) - Specification
- SAXESS XTP Protocol Specification

1.6. Abbreviations

TABLE 2

Abbreviation	Description
TCP/IP	Transmission Control Protocol/Internet Protocol
Interactive	Interactive flow
SMC	Session-based multicast flow (TCP/IP)
MC	Multicast flow
UC	Unicast flow
XTP	Exchange Transaction Protocol
XMP	Exchange Message Protocol

2. Type Definitions

This section describes the types defined by the API. Some types are included for the purpose of hiding implementation details (to increase flexibility). Others are provided to make the API more type safe (to increase stability).

2.1. Constants

Constant Name	Description
XMP_MaxMessageLength	Specifies the maximum size of a message to be returned from XM_Session_Recv. This is also the maximum size of a decompressed message.
XMP_HeaderSize	The size of the XMP header.
XMP_MaxExtHdrSize	Specifies the maximum size of the extension header.
XMP_MaxCryptKeySize	Specifies the maximum size of the crypto key.
XM_NoTimeout	Can be passed as timeout parameter to specify that an operation should not time out.
XM_NoKeyId	Can be passed to XM_Crypt_Init when initializing an XM_Crypt that will not be used in a call to XM_Session_KeyExchange (directly or indirectly through XM_Session_ConnectService or XM_Session_ConnectAddress).
XM_AnySourcePort	Can be passed to XM_Session_ConnectService or XM_Session_ConnectAddress.

2.2. Int32

Int32 defines a basic integer data type. If you already have a definition of this type, suppress redefinition in xmapi.h by defining INT32_DEFINED prior to including xmapi.h.

Definition

```
int Int32
```

2.3. XM_Timespan

XM_Timespan denotes a relative time value in seconds.

Definition

```
Int32 XM_Timespan
```

2.4. XM_SessionMode

XM_SessionMode enumerates XMAPI's modes. For "Cooked" sessions, XMAPI will automatically encrypt, decrypt and decompress messages using the corresponding function sets within the API. For applications that want to have more control, the "Raw" mode will deliver the original frames to the application layer.

Definition

```
enum XM_SessionMode
```

Members

Member	Description
XM_SessionMode_Cooked	Automatically encrypt, decrypt and decompress messages sent and received on a session.
XM_SessionMode_Raw	Deliver the original message frames.

2.5. XM_ErrorCode

XM_ErrorCode is an enumeration of standardized error codes that are returned by many API function calls.

Definition

```
enum XM_ErrorCode
```

Members

Member	Description
XME_NoError	Not an error. The last call to XM completed successfully.
XME_NoMemory	Allocation of dynamic memory failed.
XME_BadParam	A parameter supplied to a function call was invalid.
XME_BadLength	A length parameter supplied to a function call was invalid.
XME_NoRoom	No room for the operation, i.e. a table or data area is full.
XME_NoServiceEntries	No entry points have been defined for the service.
XME_NotSupported	The specific usage of the operation is not supported.
XME_NetworkError	A communications error has occurred.
XME_Timeout	The operation timed out. Not necessarily an error.
XME_BufferTooSmall	The supplied reply buffer was too small to hold the actual message.
XME_InternalError	The XM layer was unable to process the request due to an internal error.
XME_NotInitialized	XM has not been initialized.
XME_NoMoreEntries	XMP_GetExtHdrEntry cannot extract any more extension headers. Not necessarily an error.
XME_DC_EndOfFile	The decompression function set found this error in the compress input.
XME_DC_BadSize	The decompression function set found this error in the compress input.
XME_DC_BadBitStream	The decompression function set found this error in the compress input.
XME_DC_BadOpcode	The decompression function set found this error in the compress input.
XME_DC_BadHeaderType	The decompression function set found this error in the compress input.
XME_DC_BadPhraseLength	The decompression function set found this error in the compress input.
XME_DC_BadPhraseOffset	The decompression function set found this error in the compress input.
XME_BadAlgorithm	An illegal encryption algorithm was specified.
XME_BadKeyLength	The encryption key was of an unacceptable length (DES encryption only).
XME_WeakKey	The encryption key was too weak (DES encryption only).

2.6. XMP_Type

See the XMP Protocol Specification for an explanation of the types.

Definition

```
enum XMP_Type
```

Members

Member	Description
XMP_Type_None	None.
XMP_Type_XTP_2	XTP 2.
XMP_Type_BinaryRecord	Binary record.
XMP_Type_BinaryTagged	Binary tagged.
XMP_Type_KeyExchange	Key exchange.
XMP_Type_Encrypted	Encrypted.
XMP_Type_Compressed	Compressed.

2.7. XMP_Tag

See the XMP Protocol Specification for an explanation of the tags.

Definition

```
enum XMP_Tag
```

Members

Member	Description
XMP_Tag_None	None.
XMP_Tag_ClientTxnId	Client transaction ID.
XMP_Tag_ServiceTime	Service time.
XMP_Tag_KeepAlive	Keep alive.
XMP_Tag_Timestamp	Timestamp.
XMP_Tag_KeyId	Key ID.
XMP_Tag_QueueAlert	Queue alert.
XMP_Tag_CompressionAlgorithm	Compression algorithm.
XMP_Tag_EncryptionAlgorithm	Encryption algorithm.
XMP_Tag_SessionState	Session state.
XMP_Tag_ProtocolVersion	Protocol version.

2.8. XMP_CA

XMP_CA is an enumeration of supported compression algorithms.

Definition

```
enum XMP_CA
```

Members

Member	Description
XMP_CA_None	No compression.
XMP_CA_LZ1	LZ1 compression.

2.9. XMP_EA

XMP_EA is an enumeration of encryption algorithms supported by XMP.

Definition

```
enum XMP_EA
```

Members

Member	Description
XMP_EA_None	No encryption.
XMP_EA_RC4	Encryption is performed using RC4.
XMP_EA_DES	Encryption is performed using DES.

2.10. XMP_SS

XMP_SS is an enumeration of session states defined in XMP.

Definition

```
enum XMP_SS
```

Members

Member	Description
XMP_SS_None	Undefined session state.
XMP_SS_Offline	No application Logon has been performed.
XMP_SS_Catchup	Application is doing recovery.
XMP_SS_Online	Application is logged on and not doing recovery.

2.11. XMP_PV

XMP_PV is an enumeration of protocol versions defined in XMP.

Definition

```
enum XMP_PV
```

Members

Member	Description
XMP_PV_None	Undefined protocol version.
XMP_PV_Current	The current version of XMP.

2.12. XM_Service

XM_Service is an opaque representation of a service. One or more entry points can be defined for the service. XM_Service identifies the addresses to be used in XM_Session_ConnectService. The object may be used in other calls after it has successfully been initialized by XM_Service_Init. It does not become usable in connection attempts until at least one entry point has been defined using XM_Service_AddEntry.

Definition

```
struct XMI_Service* XM_Service
```

2.13. XM_Session

XM_Session is an opaque representation of a session object. The object may be used in

other calls after it has successfully been initialized by XM_Session_ConnectService or XM_Session_ConnectAddress.

Definition

```
struct XMI_Session* XM_Session
```

2.14. XM_DC

XM_DC is an opaque representation of a decompression context object. The object may be used in other calls after it has successfully been initialized by XM_DC_Init.

Definition

```
struct XMI_DC* XM_DC
```

2.15. XM_Crypt

XM_Crypt is an opaque representation of a encryption state object. The object may be used in other calls after it has successfully been initialized by XM_Crypt_Init.

Definition

```
struct XMI_Crypt* XM_Crypt
```

3. Function Interface

This section describes the functions defined in the API.

With a few exceptions, function arguments are by value or pure input arguments. Where appropriate they are declared with a const qualifier. Using const makes the API cleaner and the application programmer can rely on passed data not being modified.

3.1. Initialization

3.1.1. XM_Init

XM_Init must be called before any other XMAPI functions may be called.

Synopsis

```
XM_ErrorCode XM_Init (Int32 TextualOutputFD)
```

Arguments

TextualOutputFD	[in]	Error messages will be written to this file descriptor. If the value is negative, error messages will be suppressed.

Returns

Returns XME_NoError on success.

3.1.2. XM_InitEntropy

XM_InitEntropy initializes XMAPI with random bytes to increase entropy when XM_Session_KeyExchange is used.

The bytes in Entropy should be truly random to be of benefit. If the data is generated using a pseudo-random generator, no actual additional entropy is added.

If called, XM_InitEntropy should be called after XM_Init, but before any other calls to XMAPI.

Synopsis

```
XM_ErrorCode XM_InitEntropy (const void* Entropy,
                             Int32 Size)
```

Arguments

Entropy	[in]	Pointer to bytes with random data.
Size	[in]	The number of bytes in Entropy. More than 256 bytes are never used.

Returns

Returns XME_NoError on success.

3.2. Error Diagnostics

3.2.1. XM_GetErrorMessage

XM_GetErrorMessage returns a text string that represents the specified code. If an invalid error code is specified, XM_GetErrorMessage returns the string <Invalid XMAPI

error code>.

Note: the return type should actually be `const char* const` but at least one compiler does not accept that.

Synopsis

```
const char* XM_GetErrorMessage (XM_ErrorCode ErrorCode)
```

Arguments

ErrorCode	[in]

Returns

Returns a text string that represents the error code.

3.3. Service

3.3.1. XM_Service_Init

XM_Service_Init initializes a service object.

Synopsis

```
XM_ErrorCode XM_Service_Init (XM_Service* Service)
```

Arguments

Service	[out]	The address of a service object that will become valid on success.

Returns

Returns XME_NoError on success.

3.3.2. XM_Service_Destroy

XM_Service_Destroy frees a Service object.

XM_Service_Destroy must never be called before all session objects associated with the Service have been destroyed. N.B. This cannot be checked by XMAPI, but using a Session object associated with a destroyed Service object, for example, a reconnect attempt, will make the application crash.

Synopsis

```
XM_ErrorCode XM_Service_Destroy (XM_Service Service)
```

Arguments

Service	[in]	The service object to destroy.

Returns

Returns XME_NoError on success.

3.3.3. XM_Service_AddEntry

XM_Service_AddEntry adds an entry point for a service.

Synopsis

```
XM_ErrorCode XM_Service_AddEntry (XM_Service Service,
                                  const char* Host,
                                  Int32 Port)
```

Arguments

Service	[in]	
Host	[in]	Host name or IP address in string format, e.g. "127.0.0.1".
Port	[in]	Port number.

Returns

Returns XME_NoError on success.

3.4. Session

3.4.1. XM_Session_ConnectService

XM_Session_ConnectService connects to a service and returns when one of the entry points has been connected or when all entry points have been tried without success.

If SessionMode is XM_SessionMode_Cooked, messages received by XM_Session_Recv will be decompressed and, if OptionalCrypt is defined, decrypted automatically. Messages sent with XM_Session_Send will be encrypted if OptionalCrypt is defined. Note that key exchange messages will be sent and replied behind the scenes. To avoid internal encoding and decoding, use the XM_SessionMode_Raw mode.

Synopsis

```
XM_ErrorCode XM_Session_ConnectService (XM_Session* Session,
                                        XM_Service Service,
                                        XM_SessionMode SessionMode,
                                        XM_Crypt OptionalCrypt,
                                        Int32 SourcePort,
                                        XM_Timespan ConnectTimeout)
```

Arguments

Session	[out]	is the address of a session that will become valid on success.
Service	[in]	is the desired service to connect.
SessionMode	[in]	If "Cooked", messages that XMAPI can handle internally will be processed as much as possible, i.e encrypted, decrypted, compressed and decompressed.
OptionalCrypt	[in]	If non-NULL, OptionalCrypt should be an initialized XM_Crypt. It will be used to encrypt and decrypt the key exchange for the service, and will be destroyed by this call. Must be NULL if SessionMode is XM_SessionMode_Raw.
SourcePort	[in]	TCP source port to use. If XM_AnySourcePort is passed, the connect will use a random source port assigned by the operating system's network layer.
ConnectTimeout	[in]	The connect attempt timeout per connection attempt. This means that the total timeout for the function can be the number of service entry points multiplied by this value. If XM_NoTimeout is passed, the connect will try until the underlying TCP mechanism returns.

Returns

Returns XME_NoError on success.

3.4.2. XM_Session_ConnectAddress

XM_Session_ConnectAddress connects to the specified address.

If SessionMode is XM_SessionMode_Cooked, messages received by XM_Session_Recv will be decompressed and, if OptionalCrypt is defined, decrypted automatically. Messages sent with XM_Session_Send will be encrypted if OptionalCrypt is defined. Note that key exchange messages will be sent and replied behind the scenes. To avoid internal encoding and decoding, use the XM_SessionMode_Raw mode.

Synopsis

```
XM_ErrorCode XM_Session_ConnectAddress (XM_Session* Session,
                                        const char* Host,
                                        Int32 Port,
                                        XM_SessionMode SessionMode,
                                        XM_Crypt OptionalCrypt,
                                        Int32 SourcePort,
                                        XM_Timespan ConnectTimeout)
```

Arguments

Session	[out]	is the address of a session that will become valid on success.
Host	[in]	Host name or IP address in string format, e.g. "127.0.0.1".
Port	[in]	Port number.
SessionMode	[in]	If "Cooked", messages that XMAPI can handle internally will be processed as much as possible, i.e encrypted, decrypted and decompressed.
OptionalCrypt	[in]	If non-NULL, OptionalCrypt will be used to encrypt and decrypt the key exchange with the service. Will be destroyed (using XM_Crypt_Destroy) by this call. Must be NULL if SessionMode is XM_SessionMode_Raw.
SourcePort	[in]	TCP source port to use. If XM_AnySourcePort is passed, the connect will use a random source port assigned by the operating system's network layer.
ConnectTimeout	[in]	The connect attempt timeout. If XM_NoTimeout is passed, the connect will try until the underlying TCP mechanism returns.

Returns

Returns XME_NoError on success.

3.4.3. XM_Session_Disconnect

XM_Session_Disconnect disconnects the session and frees resources associated with the connection.

Synopsis

```
XM_ErrorCode XM_Session_Disconnect (XM_Session Session)
```

Arguments

Session	[in]

Returns

Returns XME_NoError on success.

3.4.4. XM_Session_GetAddress

XM_Session_GetAddress gets the address (host and port) information about the currently connected entry for this session.

Synopsis

```
XM_ErrorCode XM_Session_GetAddress (XM_Session Session,
                                    const char** Host,
                                    Int32* Port)
```

Arguments

Session	[in]	
Host	[out]	Will point to the host name or IP address string.
Port	[out]	The connected port number will be returned in the location referred to by Port.

Returns

Returns XME_NoError on success.

3.4.5. XM_Session_TestDistribution

Returns a positive value if the set of sessions is not distributed well over the available service entry points. For the interactive flow, XM_Session_TestDistribution can be used to avoid even a temporary single-point-of-failure in the event that a service entry point fails, i.e., without having to reconnect all interactive sessions at the time of failure.

XM_Session_TestDistribution could be called repeatedly after each reconnect until zero is returned.

It is important that the application controls when to reconnect a session, e.g., when there is no pending reply on the session.

Using XM_Session_TestDistribution is optional.

Synopsis

```
Int32 XM_Session_TestDistribution (XM_Session* Sessions,
                                   Int32 Count,
                                   XM_Session* Candidate)
```

Arguments

Sessions	[in]	A vector of sessions that must be connected using the same service.
Count	[in]	The length of the Sessions vector.
Candidate	[out]	If the return value is positive, Candidate will contain a session that would benefit from being reconnected given the current use of the service.

Returns

Returns a positive value if the sessions should be distributed better, zero if distributed well. If a negative value is returned, it indicates an error, the error code being the return value. XME_BadParam is returned unless all sessions are connected to the same service.

3.4.6. XM_Session_SetRecvTimeout

XM_Session_SetRecvTimeout sets the receive timeout value. If this function has not been called the default value, XM_NoTimeout, is used.

Synopsis

```
XM_ErrorCode XM_Session_SetRecvTimeout (XM_Session Session,
                                        XM_Timespan Timeout)
```

Arguments

Session	[in]	
Timeout	[in]	the number of seconds to wait for XM_Session_Recv calls to complete, or indefinitely if XM_NoTimeout is passed.

Returns

Returns XME_NoError on success.

3.4.7. XM_Session_Send

XM_Session_Send sends a message to the connected service. If the session is in "Cooked" mode and encryption has been enabled for it, the message will be encrypted before being sent.

Synopsis

```
XM_ErrorCode XM_Session_Send (XM_Session Session,
                              const void* Message)
```

Arguments

Session	[in]	
Message	[in]	The buffer containing the XMP message.

Returns

Returns XME_NoError on success.

3.4.8. XM_Session_Recv

XM_Session_Recv waits synchronously for a message to be received. The full XMP frame will be stored in the supplied reply buffer. XMAPI's XMP selector functions can be used to access parts of the message.

Synopsis

```
Int32 XM_Session_Recv (XM_Session Session,
                       void* Reply)
```

Arguments

Session	[in]	
Reply	[out]	The buffer that the XMP message will be stored in. The buffer must have room for XMP_MaxMessageLength bytes.

Returns

Returns received size or an error code (a negative value), which can be XME_Timeout. Never returns XME_NoError.

3.4.9. XM_Session_MultiRecv

XM_Session_MultiRecv waits synchronously for messages to be received. The full XMP

frames will be stored in the supplied reply buffers. The XMAPI's XMP selector functions can be used to access parts of the messages.

This function is useful if the application can benefit from receiving multiple messages in a single call.

Synopsis

```
Int32 XM_Session_MultiRecv (XM_Session Session,
                            void** Replies,
                            Int32 Max)
```

Arguments

Session	[in]	
Replies	[out]	The buffers that the XMP messages will be stored in. The buffers must have room for XMP_MaxMessageLength bytes each.
Max	[in]	The number of buffers pointed to by Replies.

Returns

Returns the number of replies or an error code (a negative value), which can be XME_Timeout. Never returns XME_NoError.

3.4.10. XM_Session_KeyExchange

XM_Session_KeyExchange performs key exchange with SAXESS for the specified session.

Synopsis

```
XM_ErrorCode XM_Session_KeyExchange (XM_Session Session,
                                     XM_Crypt KeyExchangeCrypt,
                                     XM_Crypt* Decrypt,
                                     XM_Crypt* Encrypt)
```

Arguments

Session	[in]	
KeyExchangeCrypt	[in]	The crypt that is used to encrypt and decrypt the key exchange. It must have been initialized using XM_Crypt_Init (with the correct KeyId) before this call.
Decrypt	[out]	The address of a crypt object that will become valid on success. This crypt object is for decrypting incoming messages.
Encrypt	[out]	The address of a crypt object that will become valid on success. This crypt object is for encrypting outgoing messages.

Returns

Returns XME_NoError on success.

3.4.11. XM_Session_SetRecvBufferSize

XM_Session_SetRecvBufferSize sets the receive buffer on the specified session.

Synopsis

```
XM_ErrorCode XM_Session_SetRecvBufferSize (XM_Session Session,
                                           Int32 BufferSize)
```

Arguments

Session	[in]	
BufferSize	[in]	The buffer size in KB.

Returns

Returns XME_NoError on success.

3.5. Decompression

3.5.1. XM_DC_Init

XM_DC_Init initializes a decompression context object.

Synopsis

```
XM_ErrorCode XM_DC_Init (XM_DC* DC,
                         XMP_CA CA)
```

Arguments

DC	[out]	The address of a context that will become valid on success.
CA	[in]	Compression algorithm

Returns

Returns XME_NoError on success.

3.5.2. XM_DC_Destroy

XM_DC_Destroy destroys a decompression context object.

Synopsis

```
XM_ErrorCode XM_DC_Destroy (XM_DC DC)
```

Arguments

DC	[in]

Returns

Returns XME_NoError on success.

3.5.3. XM_DC_Decompress

XM_DC_Decompress expands a compressed buffer. The size of the expanded data will not exceed XMP_MaxMessageLength.

Synopsis

```
Int32 XM_DC_Decompress (XM_DC DC,
                        void* OutData,
                        const void* InData,
                        Int32 InSize)
```

Arguments

DC	[in]	

OutData	[out]	The buffer that the decompressed data will be stored in. Must be at least XMP_MaxMessageLength.
InData	[in]	The buffer containing the compressed data.
InSize	[in]	The size of the compressed data.

Returns

Returns actual size of OutData buffer or an error code (a negative value) on error. Never returns XME_NoError.

3.6. Encryption

3.6.1. XM_Crypt_Init

XM_Crypt_Init initializes a crypt state object.

Synopsis

```
XM_ErrorCode XM_Crypt_Init (XM_Crypt* Crypt,
                            XMP_EA EA,
                            const void* Key,
                            Int32 KeySize,
                            Int32 KeyId)
```

Arguments

Crypt	[out]	The address of a crypt object that will become valid on success.
EA	[in]	The encryption algorithm.
Key	[in]	A buffer containing the key.
KeySize	[in]	The size of the key. At most XMP_MaxCryptKeySize bytes.
KeyId	[in]	The ID of the key or XM_NoKeyId. An actual key ID is required if the XM_Crypt is to be used in a call to XM_Session_KeyExchange, either directly or indirectly through XM_Session_ConnectService or XM_Session_ConnectAddress.

Returns

Returns XME_NoError on success.

3.6.2. XM_Crypt_Destroy

XM_Crypt_Destroy destroys an encryption context object.

Synopsis

```
XM_ErrorCode XM_Crypt_Destroy (XM_Crypt Crypt)
```

Arguments

Crypt	[in]

Returns

Returns XME_NoError on success.

3.6.3. XM_Crypt_Encrypt

XM_Crypt_Encrypt encrypts the contents of a buffer.

Synopsis

```
XM_ErrorCode XM_Crypt_Encrypt (XM_Crypt Crypt,
                               void* OutData,
                               Int32* OutSize,
                               const void* InData,
                               Int32 InSize)
```

Arguments

Crypt	[in]	
OutData	[out]	The buffer that the encrypted data will be stored in. Must have room for at least InSize bytes. If DES encryption is used, it must have room for at least InSize rounded up to the next eight byte boundary.
OutSize	[out]	If not NULL, the location pointed to will be updated with the size of the encrypted data. This may be larger than InSize due to alignment requirements (up to seven bytes if DES encryption is used).
InData	[in]	The buffer containing the plain text. If DES encryption is used, the allocated size of InData should be up to 7 bytes greater than indicated by InSize due to the block cipher nature of DES.
InSize	[in]	The size of the plain text data.

Returns

Returns XME_NoError on success.

3.6.4. XM_Crypt_Decrypt

XM_Crypt_Decrypt decrypts the contents of a buffer.

Synopsis

```
XM_ErrorCode XM_Crypt_Decrypt (XM_Crypt Crypt,
                               void* OutData,
                               const void* InData,
                               Int32 Size)
```

Arguments

Crypt	[in]	
OutData	[out]	The buffer that the plain text will be stored in. Must have room for at least Size bytes. If DES encryption is used, it must have room for at least Size rounded up to the next eight byte boundary.
InData	[in]	The buffer containing the encrypted data.
Size	[in]	The size of the data.

Returns

Returns XME_NoError on success.

3.7. XMP

3.7.1. XMP_GetBodyType

XMP_GetBodyType returns the body type of an XMP message.

Synopsis

```
Int32 XMP_GetBodyType (const void* Message)
```

Arguments

Message	[in]	Pointer to the XMP message.

Returns

The XMP message type or an error code (a negative value) on error.

Note that the type is returned as Int32 and not as XMP_Type.

3.7.2. XMP_GetFrameLen

XMP_GetFrameLen returns the size of an XMP message, including the XMP header, extension header and body.

This function is not needed by an application using "Cooked" mode.

Synopsis

```
Int32 XMP_GetFrameLen (const void* Message)
```

Arguments

Message	[in]	Pointer to the XMP message.

Returns

The XMP message size or an error code (a negative value) on error.

3.7.3. XMP_GetBody

XMP_GetBody returns a pointer to the place in an XMP message where the body starts. The size of the body is also returned in the location pointed to by BodyLen.

Synopsis

```
const void* XMP_GetBody (const void* Message,
                         Int32* BodyLen)
```

Arguments

Message	[in]	Pointer to the XMP message.
BodyLen	[out]	The size of the Body is returned at the referenced location.

Returns

A pointer to the beginning of the body within the XMP message or NULL if the message has no body. NULL is also returned if the BodyLen parameter is NULL.

3.7.4. XMP_GetNextFrame

XMP_GetNextFrame returns a pointer to the place in a buffer where the next frame starts. If NULL is returned, the buffer contains only one frame.

To iterate over the set of XMP frames in a buffer, call XMP_GetNextFrame repeatedly and provide the pointer that was returned in the previous call. For this to work, the original buffer must contain one or more complete messages.

This function is not needed by an application using "Cooked" mode.

Synopsis

```
const void* XMP_GetNextFrame (const void* Buffer,
                              Int32* RemainingLen)
```

Arguments

Buffer	[in]	A pointer to the start position of an XMP frame.
RemainingLen	[inout]	A pointer to the size of the buffer. The size will be updated to reflect the resulting size of the buffer that is returned by this function.

Returns

A pointer to the place in the buffer where the next XMP frame begins. Returns NULL if there is no more complete XMP frames.

3.7.5. XMP_GetBodyLen

XMP_GetBodyLen returns the datalength in the the xmp-header (byte 3 and 4).

Synopsis

```
Int32 XMP_GetBodyLen (const void* Buffer)
```

Arguments

Buffer	[in]	A pointer to the start position of an XMP frame.

Returns

The length in bytes is returned..

3.7.6. XMP_GetExtHeaderLen

XMP_GetExtHeaderLen returns the extended header length in the the xmp-header (byte 2).

Synopsis

```
Int32 XMP_GetExtHeaderLen (const void* Buffer)
```

Arguments

Buffer	[in]	A pointer to the start position of an XMP frame.

Returns

The length in bytes is returned..

3.7.7. XMP_GetExtHdrEntry

XMP_GetExtHdrEntry retrieves the next extension header entry of Message based on the current value of Offset.

N.B. Offset must be initalized to 0 before the first call to XMP_GetExtHdrEntry for each message.

Synopsis

```
XM_ErrorCode XMP_GetExtHdrEntry (const void* Message,
                                 Int32* Offset,
```

```
                                    XMP_Tag* Tag,
                                    const void** Data,
                                    Int32* DataLen)
```

Arguments

Message	[in]	A pointer to the start position of an XMP message.
Offset	[inout]	To get the first entry, Offset should be set to zero on the way in. To get the next entry, use the output value from the previous call
Tag	[out]	The tag of the retrieved entry will be set in the location that Tag references.
Data	[out]	The value of Data will be set to point at the start of the data of the retrieved entry. If the data is a multibyte number, (e.g. Int32) the caller is responsible for converting the data from network byte order.
DataLen	[out]	The value of DataLen will be set to the length of the entry's data.

Returns

Returns XME_NoError on success and XME_NoMoreEntries if there is no entry at the specified offset.

3.7.8. XMP_AddExtHdrEntry

XMP_AddExtHdrEntry adds an entry to an extension header buffer. The resulting buffer can be used in a subsequent call to XMP_CreateMessage.

N.B. ExtHdrLen must be initalized to 0 before the first call to XMP_AddExtHdrEntry for each buffer.

Synopsis

```
XM_ErrorCode XMP_AddExtHdrEntry (void* ExtHdr,
                                 Int32* ExtHdrLen,
                                 XMP_Tag Tag,
                                 const void* Data,
                                 Int32 DataLen)
```

Arguments

ExtHdr	[out]	A pointer to the start of the extension header buffer. The buffer must have room for XMP_MaxExtHdrSize bytes.
ExtHdrLen	[inout]	A pointer to the extension header length. On the way in, the current length. The length is updated on the way out to reflect the addition of the entry. For the first entry, the length should be set to zero on the way in.
Tag	[in]	The entry's tag type.
Data	[in]	A buffer containing the data to be set for the entry. Should be NULL if and only if the data length is zero. If the data is a multibyte number (e.g. Int32), the caller is responsible for converting the data into network byte order.
DataLen	[in]	The length of the entry data. If zero, the entry has no data.

Returns

Returns XME_NoError on success.

3.7.9. XMP_CreateMessage

XMP_CreateMessage creates an XMP frame in Buffer based on the content of the other parameters.

Synopsis

```
XM_ErrorCode XMP_CreateMessage (void* Buffer,
                                Int32 BufferSize,
                                XMP_Type Type,
                                const void* ExtHdr,
                                Int32 ExtHdrLen,
                                const void* Data,
                                Int32 DataLen)
```

Arguments

Buffer	[out]	The result buffer.
BufferSize	[in]	The allocated length of the result buffer. It should be at least XMP_HeaderSize + ExtHdrLen + DataLen.
Type	[in]	XMP body type.
ExtHdr	[in]	Pointer to a extension header. Should be NULL, if and only if ExtHdrLen is zero. An ExtHdr buffer that was created by one or more calls to XMP_AddExtHdrEntry can be supplied as this parameter.
ExtHdrLen	[in]	Length of the extension header. If zero, the extension header is empty. An ExtHdrLen value that was set by the last call to XMP_AddExtHdrEntry can be supplied as this parameter.
Data	[in]	Message data, e.g. an XTP message. Should be NULL if and only if the data length is zero. If the data is multibyte number (e.g. Int32), the caller is responsible for converting the data into network byte order.
DataLen	[in]	Data length. If zero, the message has no data.

Returns

Returns XME_NoError on success.

3.7.10. XMP_SetHeader

XMP_SetHeader sets an XMP header in Buffer based on the contents of the other parameters. This function can be used to avoid data copying in certain situations.

XMP_SetHeader is not needed by an application using "Cooked" mode.

Synopsis

```
XM_ErrorCode XMP_SetHeader (void* Buffer,
                            XMP_Type Type,
                            Int32 ExtHdrLen,
                            Int32 DataLen)
```

Arguments

Buffer	[out]	The result buffer.
Type	[in]	XMP body type.
ExtHdrLen	[in]	Length of extension header. If zero, the extension header is empty.
DataLen	[in]	Data length. If zero, the message has no data.

Returns

Returns XME_NoError on success.

Footnotes

1 one such fid per sequence number series